

Clasificación Binaria para Mantenimiento Predictivo de Turbofanés NASA FD002

Arantza Vargas, Alessandro, Mateo Ramírez

Curso CS3061 – Machine Learning

Universidad de Ingeniería y Tecnología – UTEC

Colab: <https://colab.research.google.com/drive/1os8ZEGPG55dXPs-IyawBHrtoRWZ17pue?usp=sharing>

GitHub: https://github.com/Alessandromg18/Machine_Learning_Proyecto1.git

I. RESUMEN EJECUTIVO

El presente proyecto desarrolla un modelo de clasificación binaria para mantenimiento predictivo usando el dataset NASA FD002. El objetivo fue clasificar cada registro del motor como sano o crítico a partir de condiciones operacionales, ciclos de uso y lecturas de sensores. Debido al desbalance de clases observado en el dataset, el modelo no se evaluó únicamente con accuracy, sino también con F1-score, precisión, recall, matriz de confusión y costo total esperado. Se compararon Regresión Logística, Linear SVM y Random Forest aplicando división por motor, normalización, validación cruzada y optimización de hiperparámetros. Random Forest obtuvo el mejor desempeño estadístico general, con accuracy de 0.9588 y F1-score de 0.8680. Sin embargo, al ajustar el threshold según una matriz de costos, el umbral 0.14 redujo los falsos negativos a 29, priorizando la detección de motores críticos.

II. INGENIERÍA DE DATOS Y FORMULACIÓN

El proyecto seleccionado corresponde al desafío de Mantenimiento Predictivo: Turbofanés de la NASA FD002. El objetivo es construir un modelo de clasificación binaria capaz de identificar si un motor se encuentra en condición sana o crítica a partir de sus condiciones operacionales y lecturas de sensores. De acuerdo con el enunciado, el problema se formula como una clasificación binaria donde la clase 0 representa un motor sano y la clase 1 representa un motor crítico.

El conjunto de datos utilizado está compuesto por los archivos train.csv y test.csv. El archivo train.csv contiene las observaciones etiquetadas y se utiliza para entrenar y validar los modelos. Por otro lado, el archivo test.csv no contiene la etiqueta objetivo, ya que sobre este conjunto se deben generar las predicciones finales. La variable objetivo del conjunto de entrenamiento es Crítico, la cual representa el estado del motor.

La variable objetivo se define de la siguiente manera:

Tabla I
VARIABLE OBJETIVO

Variable	Valor	Interpretación
Crítico	0	Motor sano
Crítico	1	Motor crítico

A partir de esta definición, se separaron las variables predictoras y la variable objetivo. Las variables predictoras, denotadas como X , corresponden a todas las columnas del conjunto de entrenamiento excepto Crítico. La etiqueta, denotada como y , corresponde únicamente a la columna Crítico. De esta manera, el modelo aprende una función que recibe como entrada las condiciones operacionales y sensores del motor, y devuelve como salida una predicción binaria sobre su estado.

El análisis inicial de la distribución de clases mostró un desbalance importante. Aproximadamente el 85.01 % de los registros pertenecen a la clase 0, correspondiente a motores sanos, mientras que aproximadamente el 14.99 % pertenecen a la clase 1, correspondiente a motores críticos.

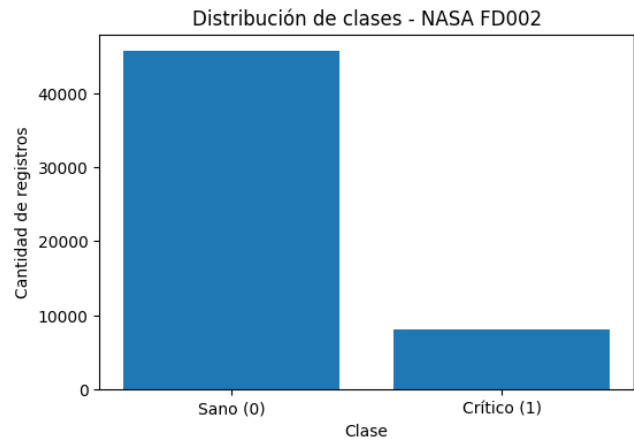


Figura 1. Distribución de clases del dataset NASA FD002.

Para la evaluación del modelo, el conjunto de entrenamiento fue dividido internamente en entrenamiento y validación. Esta partición se realizó por motor completo usando engine_id como grupo, de modo que todos los ciclos de un mismo motor quedaran únicamente en entrenamiento o únicamente en validación. Esta decisión evita data leakage, ya que un split aleatorio por filas podría colocar ciclos del mismo motor en ambos conjuntos, generando una evaluación artificialmente optimista. En este proyecto se utilizó una proporción aproximada de 80 % de motores para entrenamiento y 20 % para validación.

Como parte del Feature Engineering, se crearon variables agregadas a partir de las lecturas de sensores. Estas nuevas variables no reemplazan a los sensores originales, sino que los complementan con información resumida del comportamiento general del motor en cada ciclo. Se evitaron variables que usaran información futura, como el RUL, para no introducir data leakage en el entrenamiento.

Tabla II
FEATURE ENGINEERING

Variable creada	Descripción	Justificación
sensor_mean	Promedio de las lecturas de sensores en cada ciclo.	Resume el nivel general de las mediciones del motor.
sensor_std	Desviación estándar de las lecturas de sensores en cada ciclo.	Captura la variabilidad entre sensores, lo que puede reflejar comportamientos menos estables.
sensor_range	Diferencia entre el valor máximo y mínimo de los sensores.	Representa la dispersión extrema de las lecturas en un ciclo.

Estas variables fueron seleccionadas por su interpretación sencilla en el contexto del mantenimiento predictivo. En lugar de analizar únicamente sensores individuales, permiten representar el estado global del motor mediante medidas agregadas de nivel, variabilidad y dispersión.

Además, se aplicó normalización Z-score a las variables predictoras. Esta transformación permite que las variables queden en una escala comparable, con media cercana a cero y desviación estándar cercana a uno. El escalador fue ajustado únicamente con el conjunto de entrenamiento y luego aplicado al conjunto de validación y al conjunto de prueba. De esta manera, se evita la filtración de información del conjunto de validación o prueba hacia el entrenamiento. Este preprocesamiento es especialmente importante para modelos sensibles a la escala de las variables, como la regresión logística y SVM. En cambio, Random Forest no depende de la escala de la misma forma, porque se basa en particiones de árboles

III. METODOLOGÍA Y OPTIMIZACIÓN

Para la comparación inicial se utilizaron tres modelos de clasificación: Regresión Logística, Linear SVM y Random Forest. Buscando comparar enfoques con distintos niveles de complejidad, permitiendo evaluar si el problema se puede resolver con una separación relativamente lineal entre motores sanos y críticos o si sería más óptimo utilizar un modelo no lineal. La Regresión Logística se tomó como modelo base porque es un método interpretable para clasificación binaria y permite estimar la probabilidad de que un registro pertenezca a la clase crítica. Esto la convierte en un buen punto de partida para evaluar el desempeño de modelos más complejos.

También se incluyó Linear SVM, un modelo que busca separar las clases mediante una frontera con el mayor margen posible. Su uso es adecuado en este caso porque el dataset contiene múltiples variables numéricas, principalmente asociadas a sensores y condiciones operacionales. A diferencia de la Regresión Logística, SVM no se centra en estimar probabilidades, sino en encontrar una separación clara entre las clases.

Por último, se utilizó Random Forest, que es un método de ensamble basado en árboles de decisión. En lugar de

construir un solo árbol, este modelo entrena varios árboles y combina sus predicciones. Esto permite capturar relaciones no lineales entre las variables, algo importante en el contexto de motores turbofan, donde el estado crítico puede depender de la interacción entre varios sensores y no de una única variable aislada. Por esta razón, Random Forest permite evaluar si un modelo más flexible mejora la detección de motores críticos frente a los modelos lineales.

La comparación inicial se realizó utilizando accuracy, F1 score, precisión y recall, obteniendo los siguientes resultados:

Tabla III
COMPARACIÓN INICIAL DE MODELOS

Modelos	Accuracy	F1-score	Precisión	Recall
Regresión logística	0.938708	0.827171	0.740559	0.936725
Lineal SMV	0.942885	0.837569	0.754980	0.940447
Random Forest	0.953667	0.838638	0.813709	0.913151

La regresión logística tuvo un accuracy de 0.9387 y un F1-Score de 0.8271. Su recall fue alto, con 0.9367, lo cual implica que logró detectar gran parte de los registros críticos; sin embargo, su precisión fue la más baja, con 0.7405. Esto significa que aunque detecta muchos casos críticos también genera más falsas alarmas que los otros modelos utilizados.

El modelo lineal de SVM mejoró ligeramente los resultados de la regresión logística. Obteniendo un accuracy de 0.9428, un F1-score de 0.8375 y el recall más alto, con 0.9404, lo que significa que fue el modelo que detectó la mayor proporción de motores críticos reales. Sin embargo, su precisión fue de 0.7549, la cual es menor que la de Random Forest. Por lo tanto, aunque detecta más críticos, también produce más falsos positivos

Random Forest obtuvo una precisión de 0.8137, la más alta de los modelos. Esto quiere decir que cuando clasificó un registro como crítico, tuvo una mayor probabilidad de acertar. Su recall fue de 0.9131, menor que el de los modelos lineales, pero aún bastante alto. Es por esto que el modelo de Random Forest logró reducir las falsas alarmas sin perder demasiada capacidad para detectar motores críticos.

Además de esto se analizaron las matrices de confusión de cada modelo, las cuales permiten observar directamente cuántos casos fueron clasificados correctamente y que tipo de errores cometió cada modelo. Los falsos positivos representan motores sanos clasificados como críticos. Por el contrario, los falsos negativos representan motores críticos clasificados como sanos, lo que es más complejo en mantenimiento predictivo porque implica no detectar una posible falla.

Tabla IV
MATRICES DE CONFUSIÓN

Modelos	TN	FP	FN	TP
Regresión logística	8154	529	102	1510
Lineal SMV	8191	492	96	1516
Random Forest	8346	337	140	1472

La regresión logística detectó 1510 motores críticos y dejó sin detectar 102, sin embargo, generó 529 falsos positivos lo que significa que tiende a producir más alertas innecesarias en comparación a los otros modelos. SVM lineal presentó el menor número de falsos negativos, con 96 casos críticos no detectados, esto explica el alto recall, esto lo hace ideal si la prioridad es reducir el riesgo de pasar por alto motores críticos. Por último, el modelo de Random Forest obtuvo un menor número de falsos positivos lo que indica que fue el modelo que generó menos falsas alarmas. Esto se relaciona con su mayor precisión, sin embargo obtuvo 140 falsos negativos, lo que es más alto que los otros dos modelos.

Para analizar el equilibrio bias-variance, se comparó el desempeño de los modelos en entrenamiento y validación. Este análisis permite identificar si un modelo está sobreajustando los datos de entrenamiento o si, por el contrario, tiene poca capacidad para aprender los patrones del problema. Cuando el rendimiento en entrenamiento es mucho mayor que en validación, puede existir overfitting o alta varianza. En cambio, si ambos resultados son bajos, puede existir underfitting o alto sesgo.

En este proyecto, las diferencias entre train y validation fueron reducidas, por lo que no se observó evidencia fuerte de sobreajuste. Logistic Regression y Linear SVM presentaron comportamientos estables entre ambos conjuntos, aunque con menor capacidad para capturar relaciones complejas. Random Forest obtuvo el mayor desempeño general y presentó una diferencia moderada entre entrenamiento y validación, lo cual es esperable al tratarse de un modelo más flexible. Sin embargo, la diferencia no fue suficientemente grande como para considerar que existe overfitting severo.

Tabla V
COMPARACIÓN DE DESEMPEÑO ENTRE ENTRENAMIENTO Y VALIDACIÓN

Modelo	F1 Train	F1 Validation	Diferencia F1	Accuracy Train	Accuracy Validation	Diferencia Accuracy
Logistic Regression	0.790452	0.827171	-0.036719	0.926077	0.938708	-0.012631
Linear SVM	0.796701	0.837569	-0.040868	0.928539	0.942885	-0.014346
Random Forest	0.885313	0.860567	0.024746	0.962452	0.953667	0.008785

La tabla muestra que Random Forest obtuvo el F1-score más alto tanto en entrenamiento como en validación. Aunque su desempeño en train fue ligeramente mayor que en validation, la diferencia fue pequeña, por lo que el modelo mantiene una buena capacidad de generalización. En los modelos lineales, el F1-score de validación fue incluso mayor que el de entrenamiento, lo que indica que no existe sobreajuste y que la partición por motor generó un conjunto de validación donde estos modelos generalizaron adecuadamente.

Después de la comparación inicial, se realizó una optimización de hiperparámetros mediante validación cruzada estratificada por grupos, manteniendo la proporción de clases y evitando que un mismo motor aparezca en distintos subconjuntos de validación. La métrica principal utilizada fue el F1-score, debido al desbalance entre motores sanos y críticos, ya que esta métrica permite evaluar de forma conjunta la precisión y el recall sobre la clase crítica.

La optimización se realizó de forma independiente para cada modelo mediante validación cruzada. En Logistic Regression y Linear SVM se exploró principalmente el parámetro C, ya que este controla la regularización y la flexibilidad de la frontera de decisión. Valores pequeños de C aplican mayor regularización, generando modelos más simples, mientras que valores grandes reducen la regularización y permiten que el modelo se ajuste más a los datos. En Logistic Regression, esto limita el crecimiento de los pesos del modelo; en Linear SVM, controla la penalización de los errores de clasificación y el equilibrio entre un margen amplio y el ajuste a los datos.

Para Random Forest, la regularización no se realizó mediante C, sino controlando la complejidad de los árboles. Por ello, se evaluaron hiperparámetros como `n_estimators`, `max_depth`, `min_samples_split`, `min_samples_leaf` y `class_weight`. En particular, `max_depth`, `min_samples_split` y `min_samples_leaf` ayudan a evitar que los árboles crezcan demasiado o aprendan reglas excesivamente específicas del conjunto de entrenamiento, reduciendo así el riesgo de sobreajuste.

Luego de la búsqueda, se seleccionó para cada modelo la configuración con mejor F1-score promedio en validación cruzada, debido al desbalance entre motores sanos y críticos.

Tabla VI
HIPERPARÁMETROS EXPLORADOS DURANTE LA OPTIMIZACIÓN

Modelo	Hiperparámetros explorados
Logistic Regression	C = [0.001, 0.01, 0.1, 1, 10, 100]
Linear SVM	C = [0.001, 0.01, 0.1, 1, 10, 100]
Random Forest	<code>n_estimators</code> , <code>max_depth</code> , <code>min_samples_split</code> , <code>min_samples_leaf</code> , <code>class_weight</code>

Luego de la búsqueda, se seleccionó para cada modelo la configuración con mejor F1-score promedio en validación cruzada. En los modelos lineales, los mejores resultados se obtuvieron con menor regularización, lo que sugiere que Logistic Regression y Linear SVM necesitaban una frontera de decisión más flexible para separar adecuadamente los motores sanos y críticos.

Tabla VII
MODELOS OPTIMIZADOS

Modelos	Accuracy	F1-score	Precisión	Recall
Regresión logística	0.943371	0.838638	0.757121	0.939826
Lineal SMV	0.944148	0.840233	0.760946	0.937965
Random Forest	0.958815	0.868077	0.870787	0.865385

Después de la optimización Random Forest mantuvo el mejor desempeño estadístico general, con mayor Accuracy, F1 score y Precisión. Esto significa que fue el modelo más

equilibrado para clasificar correctamente los registros y reducir falsas alarmas, superando los modelos lineales.

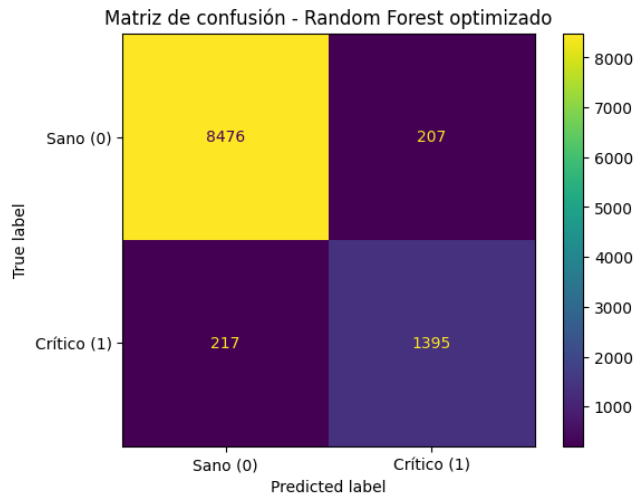


Figura 2. Matriz de confusión de Random Forest optimizado

La Figura 2 muestra el desempeño del modelo Random Forest optimizado antes del ajuste del threshold por costos. El modelo clasificó correctamente 8476 motores sanos y 1395 motores críticos. Además, generó 207 falsos positivos, lo que confirma que el modelo tiene una buena capacidad para reducir falsas alarmas. Sin embargo, también dejó 217 motores críticos sin detectar. Estos falsos negativos son importantes en mantenimiento predictivo, ya que representan motores críticos clasificados como sanos y, por lo tanto, posibles fallas no identificadas a tiempo.

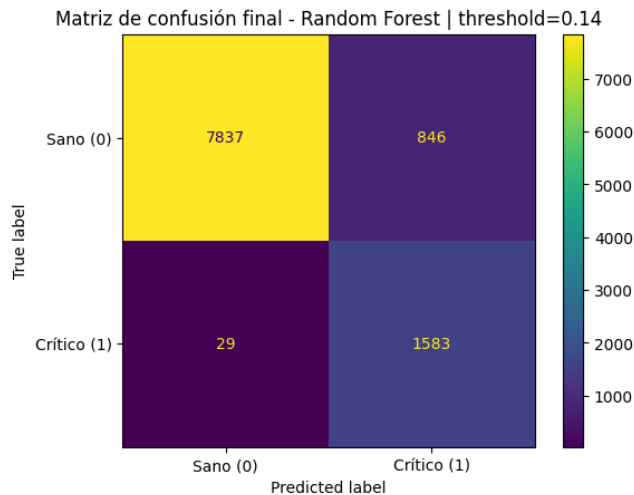


Figura 3. Matriz de confusión final de Random Forest con threshold 0.14

La Figura 3 muestra el efecto de ajustar el threshold a 0.14. Con este umbral, los falsos negativos disminuyen a 29, lo que significa que el modelo deja pasar muchos menos motores críticos como si fueran sanos. Esta reducción es relevante porque los falsos negativos son los errores más costosos

dentro de la matriz de costos definida. No obstante, el ajuste también incrementa los falsos positivos a 846, por lo que el modelo genera más falsas alarmas. En este caso, el aumento de alertas innecesarias se considera aceptable porque el objetivo principal es reducir el riesgo de no detectar motores críticos.

IV. ANÁLISIS DE COSTO-BENEFICIO

En mantenimiento predictivo, los errores del modelo no tienen el mismo impacto. Un falso positivo ocurre cuando un motor sano es clasificado como crítico, lo que puede generar una falsa alarma o una inspección innecesaria. En cambio, un falso negativo ocurre cuando un motor crítico es clasificado como sano, lo cual representa un riesgo mayor porque la falla no sería detectada a tiempo. Por ello, en este proyecto se considera más grave no identificar un motor crítico que generar una alerta preventiva adicional. Por este motivo, se definió una matriz de costos donde el falso negativo tiene una penalización mayor que el falso positivo.

Tabla VIII
MATRIZ DE COSTOS

Real / Predicho	Predice sano (0)	Predice crítico (1)
Real sano (0)	0	1000
Real crítico (1)	10000	0

Esta matriz asigna costo cero a las predicciones correctas. El falso positivo tiene un costo de 1000, asociado a una inspección o mantenimiento innecesario. El falso negativo tiene un costo de 10000, ya que implica no detectar un motor crítico a tiempo. Por ello, el análisis económico prioriza reducir los falsos negativos, incluso si esto aumenta la cantidad de falsas alarmas.

El costo total esperado se calculó como:

$$\text{Costo total} = FP \times 1000 + FN \times 10000 \quad (1)$$

Primero se identificaron los thresholds que maximizaban el F1-score. Este análisis permite observar en qué punto el modelo logra el mejor equilibrio estadístico entre precisión y recall. Los mejores resultados se muestran en la Tabla IX.

Tabla IX
THRESHOLDS CON MEJOR F1-SCORE

Threshold	Accuracy	F1-score	Precisión	Recall
0.47	0.958912	0.869886	0.862721	0.877171
0.44	0.958232	0.869539	0.850950	0.888958
0.45	0.958426	0.869512	0.854916	0.884615
0.43	0.957941	0.869224	0.846969	0.892680
0.48	0.958815	0.869055	0.865314	0.872829
0.49	0.958912	0.868919	0.868111	0.869727
0.46	0.958232	0.868421	0.856884	0.880273
0.50	0.958815	0.868077	0.870787	0.865385
0.51	0.958815	0.867500	0.874055	0.861042
0.42	0.956969	0.867087	0.839628	0.896402

La Tabla IX muestra que el mejor F1-score se obtiene con un threshold de 0.47. Este umbral alcanza un F1-score de 0.869886, con una precisión de 0.862721 y un recall de 0.877171. Esto significa que, desde una perspectiva estadística general, el threshold 0.47 ofrece el mejor equilibrio entre detectar motores críticos y evitar falsas alarmas. Sin embargo, este criterio no considera directamente el costo económico de cada tipo de error.

Posteriormente, se evaluaron los thresholds considerando la matriz de costos definida. En este caso, el objetivo ya no fue maximizar el F1-score, sino minimizar el costo total esperado, penalizando con mayor fuerza los falsos negativos.

Tabla X
THRESHOLDS ORDENADOS POR MENOR COSTO TOTAL

Th.	TN	FP	FN	TP	Costo	F1	Prec.	Recall
0.14	7837	846	29	1583	1136000	0.783469	0.651709	0.982010
0.15	7877	806	35	1577	1156000	0.789487	0.661771	0.978288
0.13	7800	883	28	1584	1163000	0.776661	0.642075	0.982630
0.16	7912	771	41	1571	1181000	0.794638	0.670794	0.974566
0.18	7970	713	48	1564	1193000	0.804320	0.686869	0.970223
0.20	8036	647	55	1557	1197000	0.816038	0.706443	0.965881
0.12	7743	940	26	1586	1200000	0.766554	0.627870	0.983871
0.17	7940	743	46	1566	1203000	0.798776	0.678216	0.971464
0.19	8007	676	53	1559	1206000	0.810502	0.697539	0.967122
0.11	7694	989	24	1588	1229000	0.758176	0.616220	0.985112

La Tabla X muestra que el menor costo total se obtiene con un threshold de 0.14. Con este umbral, el modelo alcanza un costo total de 1136000, con 846 falsos positivos y 29 falsos negativos. Aunque el threshold 0.14 no tiene el mejor F1-score, reduce considerablemente los falsos negativos, que son los errores más costosos dentro del problema. Por ello, desde el punto de vista económico y operativo, este threshold resulta más conveniente que el umbral 0.47. En cambio, el threshold 0.47 es mejor si el criterio principal es el equilibrio estadístico entre precisión y recall. Sin embargo, en mantenimiento predictivo, minimizar los falsos negativos puede ser más importante, porque no detectar un motor crítico puede generar consecuencias más graves que producir una alerta innecesaria.

Se utilizó Random forest, ya que fue el modelo con mejor desempeño general y permitió obtener probabilidades de pertenencia a la clase crítica. Por lo que primero se identificó el threshold que maximiza el F1 score. El mejor valor fue con 0.47, con un accuracy de 0.9589, F1 score de 0.8698, precisión de 0.8627 y recall de 0.8771. Este punto representa el mejor equilibrio estadístico entre precisión y recall. Sin embargo cuando se incorporó a la matriz de costos el mejor threshold fue de 0.14. Con este umbral el modelo obtuvo:

Tabla XI
THRESHOLD CON MENOR COSTO

Criterio	Threshold	TN	FP	FN	TP
Menor costo	0.14	7837	846	29	1583

$$\text{Costo total} = 846 \times 1000 + 29 \times 10000 = 1136000 \quad (2)$$

El resultado muestra que al reducir el threshold y bajar el umbral a 0.14, el modelo se vuelve más sensible para detectar motores críticos. Por eso el recall aumenta a 0.9820 y los falsos negativos disminuyen a 29. Esto es relevante, ya que los falsos negativos son los errores más costosos dentro de la matriz planeada. La desventaja sería que los falsos positivos aumentan a 846 y la precisión disminuye a 0.6517, lo que significa que el modelo genera más falsas alarmas. Sin embargo se considera mejor para lo pedido, ya que al no detectar un motor crítico tiene una penalización diez veces mayor que generar una alerta innecesaria.

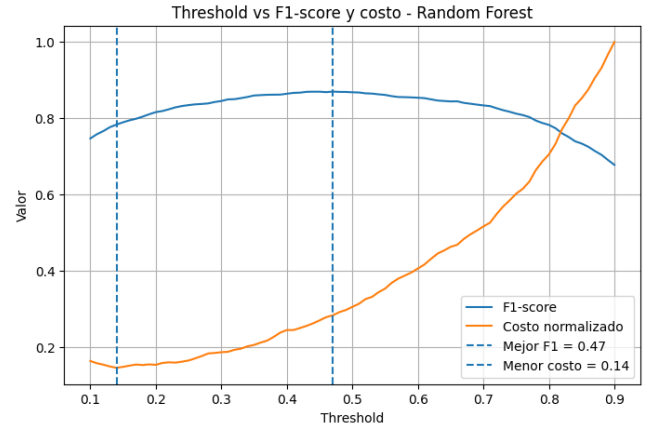


Figura 4. Relación entre threshold, F1-score y costo total en Random Forest

El gráfico muestra el efecto de modificar el threshold de decisión en el modelo Random Forest. Se observa que el F1-score alcanza su mejor valor alrededor de un umbral de 0.47, lo que representa el mejor equilibrio estadístico entre precisión y recall. Sin embargo, al incorporar la matriz de costos, el menor costo total se obtiene con un threshold cercano a 0.14. Esto ocurre porque un umbral más bajo hace que el modelo clasifique más registros como críticos, reduciendo los falsos negativos, que son los errores más costosos en el contexto de mantenimiento predictivo. Aunque esta decisión aumenta las falsas alarmas y puede reducir la precisión, resulta más conveniente desde una perspectiva operativa, ya que prioriza detectar motores críticos antes de que fallen.

V. CONCLUSIONES

El proyecto permitió construir un sistema de alerta temprana para clasificar motores NASA FD002 como sanos o críticos a partir de la base de datos de entrenamiento, implementando y comparando diferentes modelos de clasificación binaria. Como resultado, analizando la distribución de clases, se evidenció un desbalance significativo entre estas, con 85.01 % de registros y sanos y el 14.99 % de registros críticos en la base de datos. Es por esto que la evaluación no solo se limitó al accuracy, sino que se incorporaron métricas como F1-score, precisión, recall y matriz de confusión.

Después de la optimización, Random Forest obtuvo el mejor desempeño estadístico general, con accuracy de 0.9588, F1-score de 0.8680 y precisión de 0.8707. Esto indica que fue el modelo más equilibrado para clasificar correctamente los registros y reducir falsas alarmas, probablemente por su capacidad para capturar relaciones no lineales entre sensores y condiciones operacionales. Sin embargo, al incorporar la matriz de costos, el análisis cambió. Como los falsos negativos tienen mayor impacto que los falsos positivos, se ajustó el threshold a 0.14, lo que redujo los motores críticos no detectados a 29 y elevó el recall a 0.9820. Aunque esto aumentó las falsas alarmas y redujo la precisión, resulta razonable si la prioridad es evitar que un motor crítico sea clasificado como sano.

Finalmente, podemos concluir que si se busca mejorar el desempeño estadístico general Random Forest optimizado con threshold cercano a 0.47 ofrece el mejor equilibrio entre precisión y recall, por otro lado si se prioriza minimizar el costo total esperado y reducir el riesgo de no detectar motores críticos, el threshold de 0.14 es el más conveniente.

VI. REFERENCIAS Y % PARTICIPACIÓN

Participación

Tabla XII
PARTICIPACIÓN Y CONTRIBUCIONES

Integrante	Participación	Contribuciones
Alessandro	100 %	Desarrollo del Ejercicio A. Implementación manual y computacional de Regresión Lineal y Regresión Logística. Desarrollo del GD, Ecuación Normal, normalización Z-score, construcción de la matriz (X), análisis de convergencia mediante MSE, interpretación económica de coeficientes, modelos logísticos, evaluación mediante matriz de confusión, predicción y validación experimental de resultados. Además, organización general del notebook.
Mateo	100 %	Responsable del desarrollo de los primeros apartados del proyecto, relacionados con el preprocesamiento de datos, Feature Engineering, formulación inicial del problema, análisis del balance del dataset y coordinación con el TA. Sus contribuciones incluyeron la construcción y organización inicial del dataset, el análisis exploratorio de variables, la verificación de rangos y consistencia de los datos, así como la implementación de la normalización Z-score y el manejo del desbalance de clases. Además, participó en la evaluación de los tres modelos de clasificación utilizados en el proyecto: Logistic Regression, Linear SVM y Random Forest. Sus tareas incluyeron el desarrollo y ajuste de los modelos, la ejecución de procesos de Cross-Validation y optimización de hiperparámetros, la evaluación mediante métricas como Accuracy, Precision, Recall y F1-score, así como la construcción e interpretación de matrices de confusión. También contribuyó en la comparación del desempeño entre modelos, la interpretación de resultados numéricos, la comunicación de consultas al TA y la revisión constante de los avances del grupo para la estructuración final del informe.
Arantza	100 %	Responsable principal de la implementación y evaluación de los 3 modelos de clasificación utilizados en el proyecto. Sus contribuciones incluyen el desarrollo y ajuste de Logistic Regression, Linear SVM y Random Forest, ejecución de procesos de Cross - Validation y optimización de hiper parámetros, evaluación mediante métricas de Accuracy, Precision, Recall y F1-score, construcción y análisis de matrices de confusión, así como comparación de desempeño entre modelos. Además participó en la validación experimental de resultados y en la organización de tablas comparativas y visualizaciones finales.

REFERENCIAS

- [1] A. Saxena and K. Goebel, "Turbofan Engine Degradation Simulation Data Set," NASA Ames Prognostics Data Repository, 2008.
- [2] Scikit-learn Developers, "Scikit-learn: Machine Learning in Python," documentación de Logistic Regression, Linear SVM, Random Forest, model selection, preprocessing y metrics.